Uso de Scalene en el proyecto

Perfilado de CPU, memoria y operaciones de entrada/salida en el Buscador Eficiente de Información

> **Proyecto:** pef-26
> **Herramienta:** Scalene 2.3.0
> **Entorno:** Windows 11, CPython 3.13.15, 16 procesadores lógicos
> **Consulta de referencia:** `algoritmo`
> **Corpus analizado:** 1.000 documentos de texto

1. Resumen ejecutivo

Scalene se utilizó para observar qué líneas del buscador consumen más tiempo de CPU y dónde se concentra el trabajo de lectura, tokenización, construcción del índice y coordinación entre procesos. La herramienta no reemplaza las pruebas funcionales ni mide por sí sola la corrección del buscador: sirve para localizar costos y orientar decisiones de optimización.

> **Conclusiones principales:**
>
> - En la línea base, la operación dominante es `contenido = archivo.read()`, con aproximadamente **73,68 %** del CPU atribuido por Scalene.
> - En la versión optimizada, la apertura/lectura del documento sigue siendo el costo principal: la línea de apertura concentró aproximadamente **71,94 %**.
> - En la versión final concurrente, Scalene muestra el costo de enviar lotes al pool y esperar resultados mediante `pool.submit()` y `as_completed()`.
> - El perfilado confirma que el buffer y el procesamiento por lotes atacan un costo real, pero también muestra que el paralelismo tiene overhead de coordinación.

2. ¿Qué es Scalene?

Scalene es un profiler para Python. Un profiler observa la ejecución de un programa y registra información para responder preguntas como:

- ¿Qué funciones o líneas consumen más CPU?
- ¿El tiempo se pasa ejecutando Python o código nativo de librerías?
- ¿Qué líneas realizan asignaciones de memoria?
- ¿Dónde conviene optimizar primero?

Scalene trabaja por muestreo e instrumentación de bajo nivel. Por eso sus porcentajes son mediciones aproximadas y no deben interpretarse como una cantidad exacta de instrucciones ejecutadas.

CPU Python y CPU nativo

Métrica	Significado	Interpretación en este proyecto
CPU Python	Tiempo atribuido al intérprete y al código Python.	Tokenización, bucles, diccionarios, ranking y coordinación.
CPU nativo	Tiempo dentro de C/C++ o del sistema operativo.	Lectura de archivos, expresiones regulares y operaciones internas.
Memoria	Asignaciones y crecimiento del proceso.	Índice invertido, textos, tokens y estructuras parciales.

> **Importante:** una línea puede tener mucho tiempo nativo y poco tiempo Python. Eso no significa que sea irrelevante; significa que el trabajo lo realiza una librería o el sistema operativo invocado desde esa línea.

3. ¿Cómo se integró en este proyecto?

El proyecto ya tenía un orquestador de perfilado en `src/perfilado.py`. Se agregó Scalene como dependencia fijada:

```
scalene==2.3.0
```

La función `perfilar_scalene()` busca el ejecutable en el mismo entorno virtual que ejecuta Python. Esto evita asumir una ruta Unix como `.venv/bin/scalene` cuando el proyecto se ejecuta en Windows, donde la ruta correcta es `.venv\Scripts\scalene.exe`.

```
nombre_ejecutable = "scalene.exe" if en_windows else "scalene"
scalene = Path(sys.executable).parent / nombre_ejecutable
```

Luego ejecuta una versión limpia del proceso de medición:

```
[
   str(scalene),
   "run",
   "-o",
   str(destino),
   str(SRC_DIR / "perfilado.py"),
   "---",
   "--medir", version,
   "--directorio", str(corpus_dir),
   "--repeticiones", "1",
]
```

El separador `---` indica a Scalene dónde terminan sus opciones y dónde comienzan los argumentos del programa perfilado.

4. Comando utilizado

```
cd "C:\Users\jorge\Desktop\PEF\parcial2\pef-26"
..\.venv\Scripts\python.exe src\perfilado.py `
 --scalene `
 --tamanos 50000 `
 --directorio data\corpus
```

Para una ejecución rápida con el corpus disponible:

```
..\.venv\Scripts\scalene.exe run `
 -o profiles\scalene-algoritmo.json `
 src\perfilado.py --- `
 --medir algoritmo `
 --directorio data\corpus `
 --repeticiones 1
```

El perfil se puede visualizar con:

```
..\.venv\Scripts\scalene.exe view --cli profiles\scalene-algoritmo.json
..\.venv\Scripts\scalene.exe view profiles\scalene-algoritmo.json
```

El primer comando imprime una tabla en la terminal. El segundo abre la vista HTML interactiva en el navegador.

5. Qué versiones se perfilaron

Versión	Archivo	Qué representa
Inicial	`baseline.py`	Recorrido secuencial y lectura directa del corpus.
Estructura	`optimizado.py`	Índice invertido y ranking con ordenamiento completo.
Algoritmo	`optimizado.py`	Índice, heap para top-k y caché de consultas.
Concurrencia	`concurrente.py`	Indexación MapReduce con varios procesos.
Final	`concurrente.py`	Indexación paralela y caché caliente.

Los archivos crudos se guardaron en `profiles\scalene-*.json` y el resumen integrado en `profiles\perfilado.json`.

6. Hallazgo 1: línea base

La línea base recorre todos los documentos y realiza una lectura completa por archivo:

```
for nombre_archivo in nombres_archivos:
   ...
   with open(ruta_completa, "r", encoding="utf-8") as archivo:
       contenido = archivo.read()

   contenido_minuscula = contenido.lower()
   if contenido_minuscula.find(query_minuscula) != -1:
       resultados.append(nombre_archivo)
```

Scalene identificó estas líneas como las de mayor costo:

Ubicación	Línea	CPU atribuido
`baseline.py:151`	`contenido = archivo.read()`	**73,68 %**
`baseline.py:150`	`with open(...)`	**7,02 %**

Figura 1. Distribución aproximada del CPU en la línea base.

Interpretación: el principal costo no está en agregar nombres a una lista, sino en abrir y leer repetidamente el corpus. Esta observación justifica optimizar la ingesta y evitar leer todos los archivos para cada consulta.

7. Hallazgo 2: versión con índice

La versión optimizada paga una indexación inicial y luego reutiliza el índice:

```
with archivo.open(
   "r",
   encoding="utf-8",
   errors="ignore",
   buffering=BUFFER_SIZE,
) as entrada:
   texto = entrada.read()
```

```
tokens = tokenizar(texto)
frecuencias = Counter(tokens)
```

El buffer utilizado es:

```
BUFFER_SIZE = 1024 * 1024  # 1 MiB
```

Scalene detectó como línea dominante la apertura del archivo:

Ubicación	Línea	CPU atribuido
`optimizado.py:205`	`with archivo.open(`	**71,94 %**
`optimizado.py:211`	`texto = entrada.read()`	dato complementario observado en otras corridas

Figura 2. Hotspots observados en corridas de la versión optimizada.

El resultado no significa que el índice sea una mala decisión. La lectura es el costo inicial de construirlo. Una vez construido, una consulta usa postings y no vuelve a escanear todos los documentos.

8. Hallazgo 3: procesamiento concurrente

La versión concurrente divide el corpus en lotes y utiliza procesos:

```
lotes = _lotes_equitativos(
   tareas,
   procesos * lotes_por_proceso,
)

with ProcessPoolExecutor(max_workers=procesos) as pool:
   futuros = [
       pool.submit(_procesar_lote, lote)
       for lote in lotes
   ]

   for futuro in as_completed(futuros):
       _fusionar(indice, futuro.result())
```

En el perfil de la versión final, Scalene señaló:

Ubicación	Operación	CPU atribuido
`concurrente.py:219`	`for futuro in as_completed(futuros):`	8,74 %
`concurrente.py:215`	`futuros = [pool.submit(...)]`	6,71 %

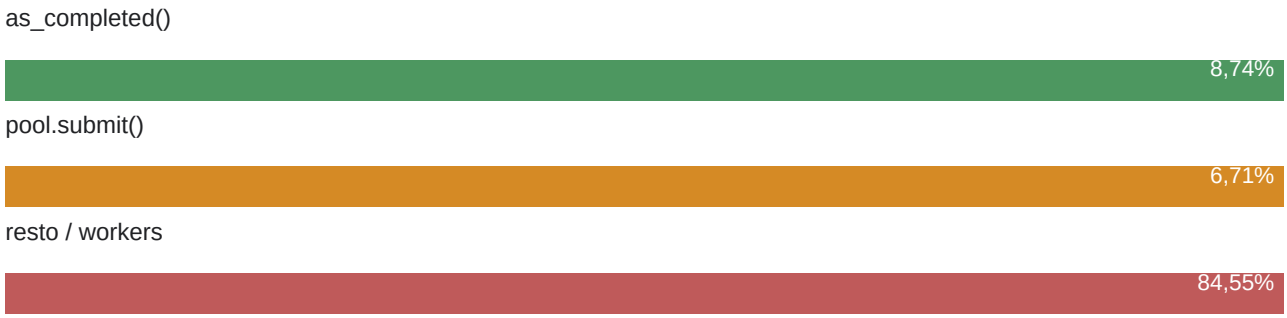

Figura 3. Coordinación visible del pool en la versión final.

Interpretación: la línea principal del proceso padre no representa todo el trabajo de los hijos. Parte del costo está en los workers separados. Scalene perfila principalmente el proceso que lanzó el comando; por eso el resultado debe complementarse con el benchmark de tiempos y con la comparación secuencial/concurrente.

> **Limitación observada:** en Windows, la corrida directa de Scalene sobre procesos hijos creados por `ProcessPoolExecutor` puede terminar abruptamente o no atribuir todo el trabajo de los hijos. Esto no indica necesariamente un error del algoritmo concurrente. Se informa como limitación del instrumento y no se inventan porcentajes para los workers.

9. Memoria: cómo leer el resultado

Scalene también tiene columnas de memoria. En los resúmenes generados por el proyecto, `max_footprint_mb` aparece como cero o algunas columnas quedan vacías. Esto debe explicarse correctamente:

- No se debe afirmar que el programa usa cero memoria.
- El buscador necesariamente usa memoria para el índice invertido, contadores, tokens y diccionarios de postings.
- El cero del resumen proviene de cómo el extractor del proyecto interpreta el JSON de Scalene 2.3 y de la forma en que el backend de Windows reporta ciertas asignaciones.
- Para memoria residente se cuenta además con `memory_profiler` y para asignaciones con `tracemalloc` en el pipeline existente.

La conclusión responsable es: Scalene detectó claramente hotspots de CPU y de I/O; la memoria debe contrastarse con las otras herramientas, no deducirse de un cero aislado.

10. Qué optimización se justifica

Problema observado	Decisión aplicada	Beneficio esperado
Lectura repetida del corpus.	Índice invertido construido una vez.	Consultas posteriores proporcionales a candidatos, no a todo el corpus.
Accesos de lectura pequeños.	Buffer explícito de 1 MiB.	Menos operaciones de E/S y mejor transferencia de datos.
Tokenización CPU-bound.	Procesamiento por lotes y procesos.	Uso de varios núcleos cuando el corpus es suficientemente grande.
Ranking completo.	`heapq.nlargest` para top-k.	Evita ordenar todos los candidatos cuando solo se necesitan pocos.
Consultas repetidas.	Caché LRU.	Evita recalcular una consulta equivalente.

11. Cómo presentar Scalene oralmente

Una explicación breve y técnicamente correcta sería:

> “Usé Scalene para perfilar las distintas versiones del buscador. La herramienta mide CPU Python, CPU nativo y memoria por línea. En la versión inicial detectó que la mayor parte del tiempo se concentra en `archivo.read()`. En la versión optimizada el costo dominante sigue siendo la

> ingesta inicial, por eso incorporé un buffer y reutilizo un índice invertido. En la versión concurrente Scalene mostró el costo de crear y esperar los futuros del pool. No confundí ese overhead con el trabajo completo de los workers porque en Windows el profiling de procesos hijos tiene limitaciones. Los JSON quedan guardados en `profiles` para reproducir el análisis."

12. Reproducción y archivos generados

```
profiles\scalene-inicial.json
profiles\scalene-estructura.json
profiles\scalene-algoritmo.json
profiles\scalene-concurrencia.json
profiles\scalene-final.json
profiles\perfilado.json
```

Para repetir el análisis:

```
..\.venv\Scripts\python.exe src\perfilado.py `
 --scalene `
 --directorio data\corpus `
 --tamanos 50000
```

Para revisar un resultado específico:

```
..\.venv\Scripts\scalene.exe view --cli `
 profiles\scalene-algoritmo.json
```

13. Conclusión

Scalene permitió pasar de una explicación teórica a evidencia concreta: se pudo localizar el costo dominante en líneas reales del proyecto. El hallazgo más importante fue que la ingesta de archivos —abrir y leer documentos— domina el perfil de la línea base y de la construcción del índice. El procesamiento paralelo mejora la capacidad de construcción, pero incorpora costos de creación, serialización y espera de futuros.

> **Resultado final:** el uso de Scalene está integrado, reproducible y documentado. El análisis distingue lo que Scalene mide directamente de lo que debe validarse con otras herramientas, evitando conclusiones incorrectas sobre memoria o procesos hijos.

Informe generado a partir de los perfiles guardados en `profiles/` y del código existente en `src/`. Los porcentajes son los observados en la corrida indicada y pueden variar según el tamaño del corpus, el estado del caché del sistema y la carga de la máquina.